14

Không mất, không trùng

At-most-once · at-least-once · exactly-once

Hôm nay mình nói gì

1. 3 delivery semantics
2. Idempotent producer
3. Transaction
4. Dead-letter queue
5. Schema registry & Avro

3 delivery semantics

	At-most-once	At-least-once mặc định thực tế	Exactly-once
Có thể mất?	Có	Không	Không
Có thể trùng?	Không	Có	Không
Cách đạt	Commit trước khi xử lý	Commit sau khi xử lý	Idempotent + transaction
Chi phí	Thấp nhất	Thấp	Cao, chậm hơn
Dùng khi	Metric không quan trọng	Mặc định, consumer idempotent	Tiền, tồn kho

Trùng xảy ra ở đâu

ĐẦU GHI · PRODUCER

Producer | **Broker**

1 · gửi msg → broker ghi xong

2 · ack về bị **mất trên mạng**

3 · producer không thấy ack → **gửi lại**

→ **broker có 2 bản**

ĐẦU ĐỌC · CONSUMER

Consumer | **Postgres**

1 · đọc batch, ghi vào DB xong

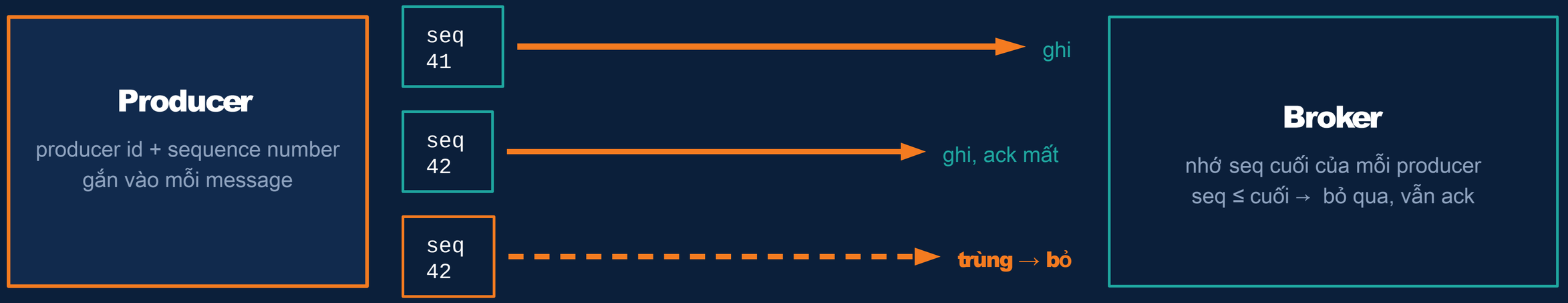
2 · **chết** trước khi commit offset

3 · khởi động lại → **đọc lại** từ offset cũ

→ **DB có 2 bản**

Trùng ở cả hai đầu, không chỉ một.

Idempotent producer



`enable.idempotence=true` — bật mặc định từ Kafka 3.

Exactly-once có thật không?

- Kafka → Kafka: có, bằng transaction (read-process-write)
- Kafka → hệ ngoài (DB, S3): không tự có
- Thực tế: at-least-once + consumer idempotent (upsert, ghi đè partition)
- "Effectively-once"

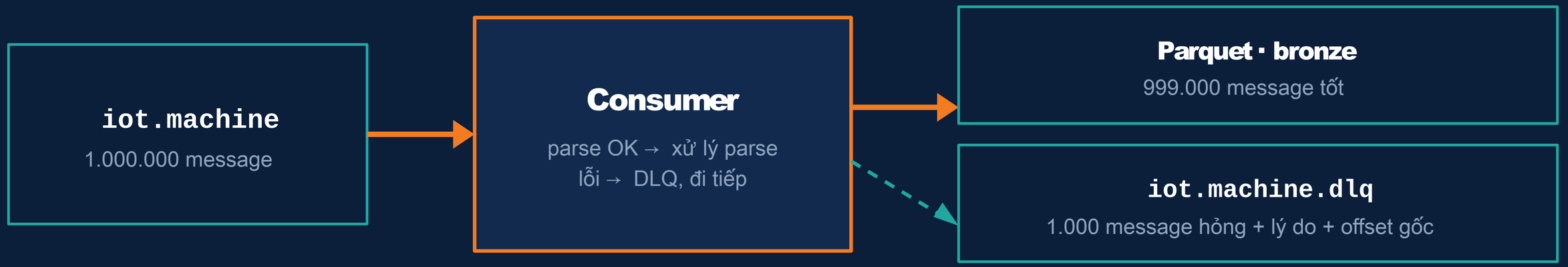
Consumer idempotent

```
UPSERT = """
    INSERT INTO machine_events (event_id, machine_id, ts, status)
    VALUES (%(event_id)s, %(machine_id)s, %(ts)s, %(status)s)
    ON CONFLICT (event_id) DO NOTHING
"""

while True:
    msgs = c.consume(num_messages=500, timeout=1.0)
    if not msgs: continue
    with pg.cursor() as cur:
        cur.executemany(UPSERT, [json.loads(m.value()) for m in msgs])
    pg.commit()
    c.commit(asynchronous=False)    # offset sau khi DB commit
```

Khoá sự kiện là chìa khoá của idempotent consumer. Đọc lại = không sao.

Dead-letter queue



Một message hỏng không được chặn 1 triệu message tốt.

Schema registry

- Producer và consumer thống nhất schema qua registry
- Avro nhỏ hơn JSON, có schema evolution
- Compatibility `BACKWARD`: thêm cột có default
- Đổi schema phá tương thích → registry từ chối

Avro schema + registry

```
SCHEMA = """{
  "type": "record", "name": "machine_event",
  "fields": [
    {"name": "event_id",   "type": "string"},
    {"name": "machine_id", "type": "string"},
    {"name": "status",     "type": "string"},
    {"name": "water_temp", "type": ["null", "float"], "default": null}
  ]
}"""

registry = SchemaRegistryClient({"url": "http://schema-registry:8081"})
serializer = AvroSerializer(registry, SCHEMA)
p.produce("iot.machine", key=ev["machine_id"], value=serializer(ev, ctx))
```

Có default = consumer cũ vẫn đọc được.

Lỗi hay gặp

SAI	ĐÚNG
~~Tin "exactly-once" của vendor, không upsert~~	Consumer luôn idempotent
~~Bỏ message lỗi im lặng~~	DLQ + alert
~~JSON không schema cho topic sống lâu~~	Avro / Protobuf + registry
~~Đổi kiểu cột trong schema~~	Thêm cột mới, deprecate cột cũ

Lab #14 — DLQ và consumer chết giữa chừng

1. Thêm schema registry + Avro cho topic iot.machine
2. Thêm DLQ, bơm 5% hỏng: `coffeegen stream --corrupt 0.05`
3. Kill consumer giữa batch, khởi động lại
4. Đếm số dòng Parquet vs số message gửi

KẾT QUẢ MONG ĐỢI

Không trùng, không mất

DLQ nhận đúng 5%

CÂU HỎI PHỎNG VẤN #14

Exactly-once có thật không?

Trong Kafka có với transaction; ra hệ ngoài cần consumer idempotent.

VIDEO TIẾP THEO

15 Stream Processing: Spark Structured Streaming vs Flink, watermark, window

Thứ Ba 22/12 · 19:00

Đăng ký